Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU) ISO 21001:2018 Certified by TUV

DBMS MINI PROJECT REPORT

CIVIC VOICE

AI-Powered Citizen Service Platform

"Bridging the gap between citizens and the services that matter most."

Flutter	AWS Bedrock	DynamoDB	Meta Llama 3

SUBJECT: Database Management System Laboratory

SUBJECT CODE: BIT24PC04

SUBMITTED BY:

Name	PRN
Om Bharambe	124B1F001
Atharva Lokhande	124B1F009
Sidhant Mattoo	124B1F016
Madhav Khobare	124B1F023

UNDER THE GUIDANCE OF:

Mrs. Alpana Borse

ACADEMIC YEAR: 2025-2026

Abstract

Civic Voice (CVI) is an AI-powered citizen service platform designed to bridge the gap between Indian citizens and the complex ecosystem of government welfare schemes and public services. In India, many eligible citizens fail to access benefits due to fragmented portals, language barriers, and lack of proper guidance. This project addresses the issue by developing a relational database system that manages citizen profiles, government schemes, services, documents, and application tracking within a unified structure. The database is normalized to Third Normal Form (3NF) to ensure data integrity and reduce redundancy. It incorporates SQL operations such as DDL, DML, joins, aggregations, subqueries, and views to support efficient data handling. Integrated with AWS cloud services like DynamoDB, AppSync, Cognito, and Bedrock, the system combines relational and NoSQL strengths. The platform supports SDGs 9, 11, and 16 by enhancing digital infrastructure, inclusivity, and transparent governance.

Introduction

Civic Voice (CVI) is a digital platform designed to bridge the gap between Indian citizens and the complex ecosystem of government services and welfare schemes. In India, hundreds of government schemes exist at central and state levels, yet millions of eligible citizens remain unaware or unable to access them due to language barriers, lack of information, and complex paper-based processes. Civic Voice addresses this by providing a centralized, AI-powered, voice-first platform that allows citizens to discover schemes, check eligibility, manage documents, and apply for services — all from a single mobile application.

The database system underpinning Civic Voice manages citizen profiles, government services, scheme eligibility, document records, and application statuses. This mini project demonstrates the design of a normalized relational schema and the implementation of SQL queries to support the core functionalities of the platform, applying DBMS concepts including entity-relationship modeling, normalization, joins, aggregate functions, views, and subqueries.

Objectives

- To design a database system for Civic Voice — AI-Powered Citizen Service Platform.
- To implement SQL queries using database management concepts.
- To model entities such as Citizens, Government Schemes, Services, Documents, and Applications with proper relationships.
- To apply normalization (1NF, 2NF, 3NF) to ensure data integrity and eliminate redundancy.
- To implement SELECT, INSERT, UPDATE, DELETE, JOIN, GROUP BY, subqueries, and VIEWs.
- To generate meaningful reports using aggregate functions (COUNT, AVG, SUM).

Database Design

ER Diagram

The ER diagram below represents the five key entities and their relationships within the Civic Voice database:

Tables and Attributes

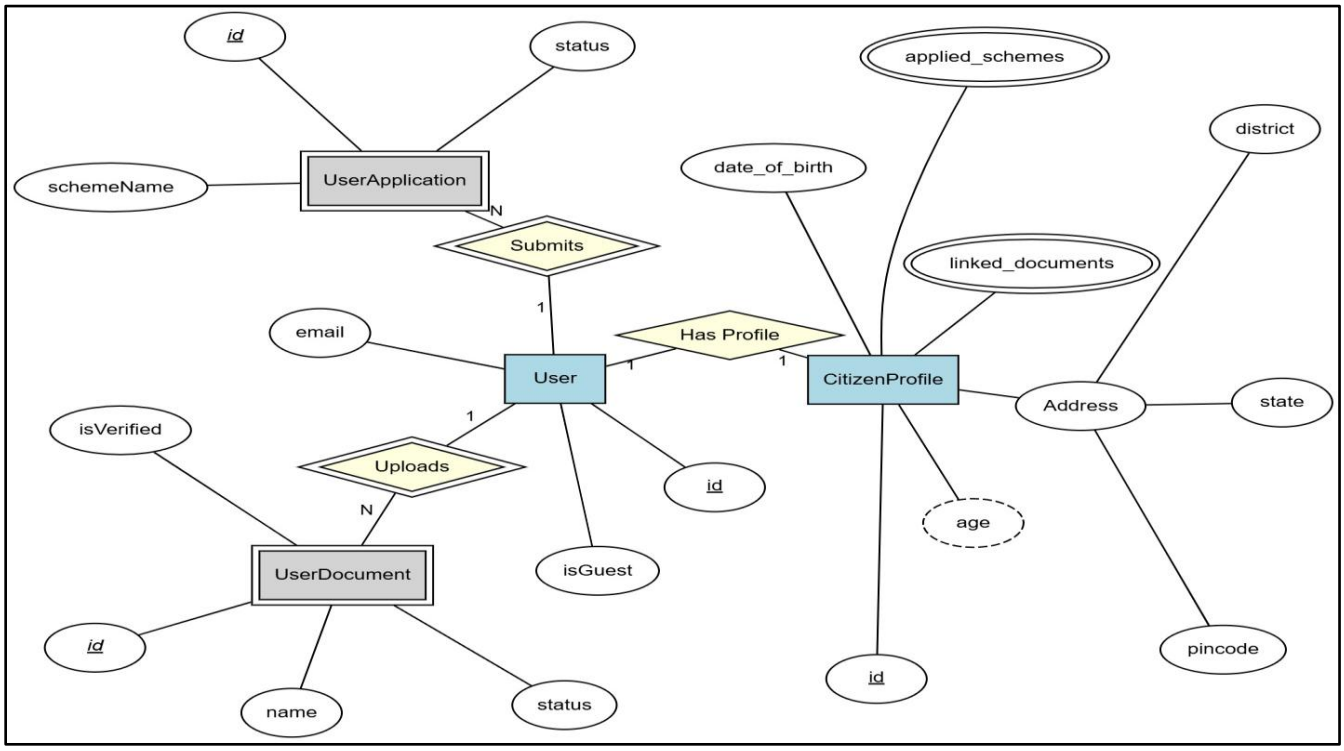


TABLE NAME	KEY ATTRIBUTES	DESCRIPTION
CITIZEN	citizen_id (PK), name, dob, gender, state, district, language, income_level, phone	Stores registered citizen profile data
GOVERNMENT_SCHEME	scheme_id (PK), name, department, eligibility_criteria, benefit_amount, deadline	Central & state government welfare schemes
SERVICE	service_id (PK), name, type, fee, processing_days, official_url	Public services: Aadhaar, PAN, Passport etc.
DOCUMENT	doc_id (PK), citizen_id (FK), doc_type, file_path, upload_date, status	Citizen documents stored in Smart Vault
APPLICATION	app_id (PK), citizen_id (FK), scheme_id (FK), status, applied_date, remarks	Tracks citizen applications for schemes

Primary Keys and Foreign Keys

Key Constraints

PK CITIZEN.citizen_id — Uniquely identifies each citizen
PK GOVERNMENT_SCHEME.scheme_id — Uniquely identifies each scheme
PK SERVICE.service_id — Uniquely identifies each service
PK DOCUMENT.doc_id — Uniquely identifies each document
PK APPLICATION.app_id — Uniquely identifies each application

FK DOCUMENT.citizen_id — References CITIZEN(citizen_id)
FK APPLICATION.citizen_id — References CITIZEN(citizen_id)
FK APPLICATION.scheme_id — References GOVERNMENT_SCHEME(scheme_id)

Implementation — SQL Queries

1. CREATE TABLE Statements

```sql
-- ─── Table 1: CITIZEN ──────────────────────────────────────────────────────
CREATE TABLE CITIZEN (
    citizen_id    INT          PRIMARY KEY AUTO_INCREMENT,
    name          VARCHAR(100) NOT NULL,
    dob           DATE,
    gender        VARCHAR(10),
    state         VARCHAR(50),
    district      VARCHAR(50),
    language      VARCHAR(30),
    income_level  VARCHAR(20),
    phone         VARCHAR(15)  UNIQUE
);

-- ─── Table 2: GOVERNMENT_SCHEME ────────────────────────────────────────────
CREATE TABLE GOVERNMENT_SCHEME (
    scheme_id           INT          PRIMARY KEY AUTO_INCREMENT,
    name                VARCHAR(150)  NOT NULL,
    department          VARCHAR(100),
    eligibility_criteria TEXT,
    benefit_amount      DECIMAL(12,2),
    deadline            DATE
);

-- ─── Table 3: SERVICE ──────────────────────────────────────────────────────
CREATE TABLE SERVICE (
    service_id      INT          PRIMARY KEY AUTO_INCREMENT,
    name            VARCHAR(100) NOT NULL,
    type            VARCHAR(50),
    fee             DECIMAL(10,2),
    processing_days INT,
    official_url    VARCHAR(255)
);
```

```sql
-- ─── Table 4: DOCUMENT ─────────────────────────────────────────────────────
CREATE TABLE DOCUMENT (
    doc_id      INT          PRIMARY KEY AUTO_INCREMENT,
    citizen_id  INT,
    doc_type    VARCHAR(50),
    file_path   VARCHAR(255),
    upload_date DATE,
    status      VARCHAR(20)  DEFAULT 'Verified',
    FOREIGN KEY (citizen_id) REFERENCES CITIZEN(citizen_id)
);

-- ─── Table 5: APPLICATION ──────────────────────────────────────────────────
CREATE TABLE APPLICATION (
    app_id      INT          PRIMARY KEY AUTO_INCREMENT,
    citizen_id  INT,
    scheme_id   INT,
```

```
    status       VARCHAR(30)  DEFAULT 'Pending',
    applied_date DATE,
    remarks      TEXT,
    FOREIGN KEY (citizen_id) REFERENCES CITIZEN(citizen_id),
    FOREIGN KEY (scheme_id)  REFERENCES GOVERNMENT_SCHEME(scheme_id)
);
```

2. INSERT Queries

```
-- Insert citizen records
INSERT INTO CITIZEN (name, dob, gender, state, district, language, income_level, phone)
VALUES ('Madhav Khobare', '2004-05-15', 'Male',
       'Maharashtra', 'Pune', 'Hindi', 'Below 2.5L', '9876543210');

INSERT INTO CITIZEN (name, dob, gender, state, district, language, income_level, phone)
VALUES ('Priya Sharma', '1990-08-22', 'Female',
       'Maharashtra', 'Nashik', 'Marathi', 'Below 1.5L', '9123456789');

-- Insert scheme records
INSERT INTO GOVERNMENT_SCHEME (name, department, eligibility_criteria, benefit_amount, deadline)
VALUES ('PM-KISAN', 'Agriculture Ministry',
       'Small & marginal farmers with land holding', 6000.00, '2025-12-31');

INSERT INTO GOVERNMENT_SCHEME (name, department, eligibility_criteria, benefit_amount, deadline)
VALUES ('Ayushman Bharat', 'Health Ministry',
       'Annual income below 2.5 lakh', 500000.00, '2026-03-31');

-- Insert application record
INSERT INTO APPLICATION (citizen_id, scheme_id, status, applied_date, remarks)
VALUES (1, 2, 'Pending', CURDATE(), 'Applied via CVI platform');
```

3. SELECT Queries with JOINs and Aggregation

```
-- Q1: All citizens with their application status and scheme name
SELECT C.name, C.state, G.name AS scheme_name, A.status, A.applied_date
FROM   CITIZEN C
JOIN   APPLICATION A        ON C.citizen_id = A.citizen_id
JOIN   GOVERNMENT_SCHEME G   ON A.scheme_id  = G.scheme_id;

-- Q2: Count total applications received per scheme
SELECT G.name AS scheme_name, COUNT(A.app_id) AS total_applications
FROM   GOVERNMENT_SCHEME G
LEFT   JOIN APPLICATION A ON G.scheme_id = A.scheme_id
GROUP  BY G.name
ORDER  BY total_applications DESC;

-- Q3: Pending applications from Maharashtra citizens
SELECT C.name, C.district, A.status, A.applied_date
FROM   CITIZEN C
JOIN   APPLICATION A ON C.citizen_id = A.citizen_id
```

```
WHERE  C.state = 'Maharashtra' AND A.status = 'Pending';

-- Q4: Citizens who have NOT applied for any scheme (Subquery)
SELECT name, phone, state
FROM   CITIZEN
WHERE  citizen_id NOT IN (SELECT DISTINCT citizen_id FROM APPLICATION);
```

4. UPDATE and DELETE Queries

```
-- Approve a pending application after document verification
UPDATE APPLICATION
SET    status = 'Approved',
       remarks = 'Documents verified successfully'
WHERE  app_id = 1;

-- Increase PM-KISAN benefit amount by 10%
UPDATE GOVERNMENT_SCHEME
SET    benefit_amount = benefit_amount * 1.10
WHERE  name = 'PM-KISAN';

-- Delete old rejected applications
DELETE FROM APPLICATION
WHERE  status = 'Rejected'
AND    applied_date < '2024-01-01';
```

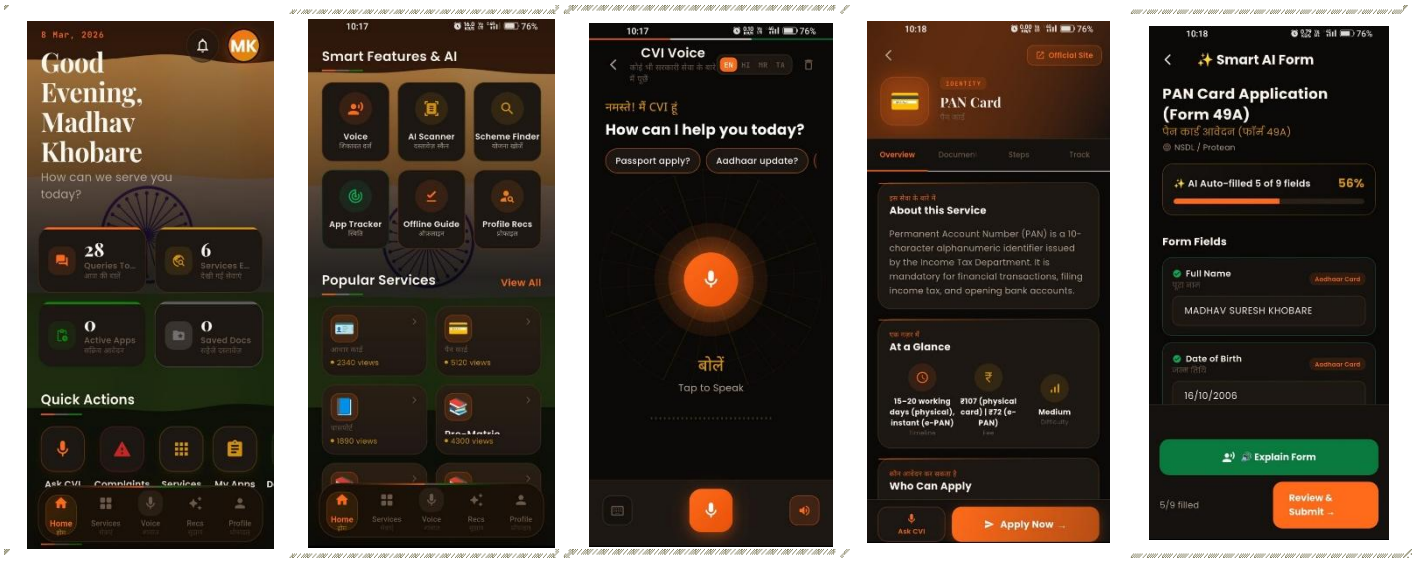
5. VIEW and Advanced Aggregate Query

```
-- Create a VIEW for citizen-scheme application summary
CREATE VIEW CVI_ApplicationSummary AS
SELECT C.citizen_id, C.name AS citizen_name, C.state,
       G.name AS scheme, A.status, A.applied_date
FROM   CITIZEN C
JOIN   APPLICATION A        ON C.citizen_id = A.citizen_id
JOIN   GOVERNMENT_SCHEME G   ON A.scheme_id  = G.scheme_id;

-- Query the view — all approved applications
SELECT * FROM CVI_ApplicationSummary WHERE status = 'Approved';

-- Average benefit amount and scheme count grouped by department
SELECT department,
       AVG(benefit_amount)  AS avg_benefit,
       COUNT(*)             AS scheme_count,
       SUM(benefit_amount)  AS total_benefit
FROM   GOVERNMENT_SCHEME
GROUP  BY department
HAVING AVG(benefit_amount) > 10000
ORDER  BY avg_benefit DESC;
```

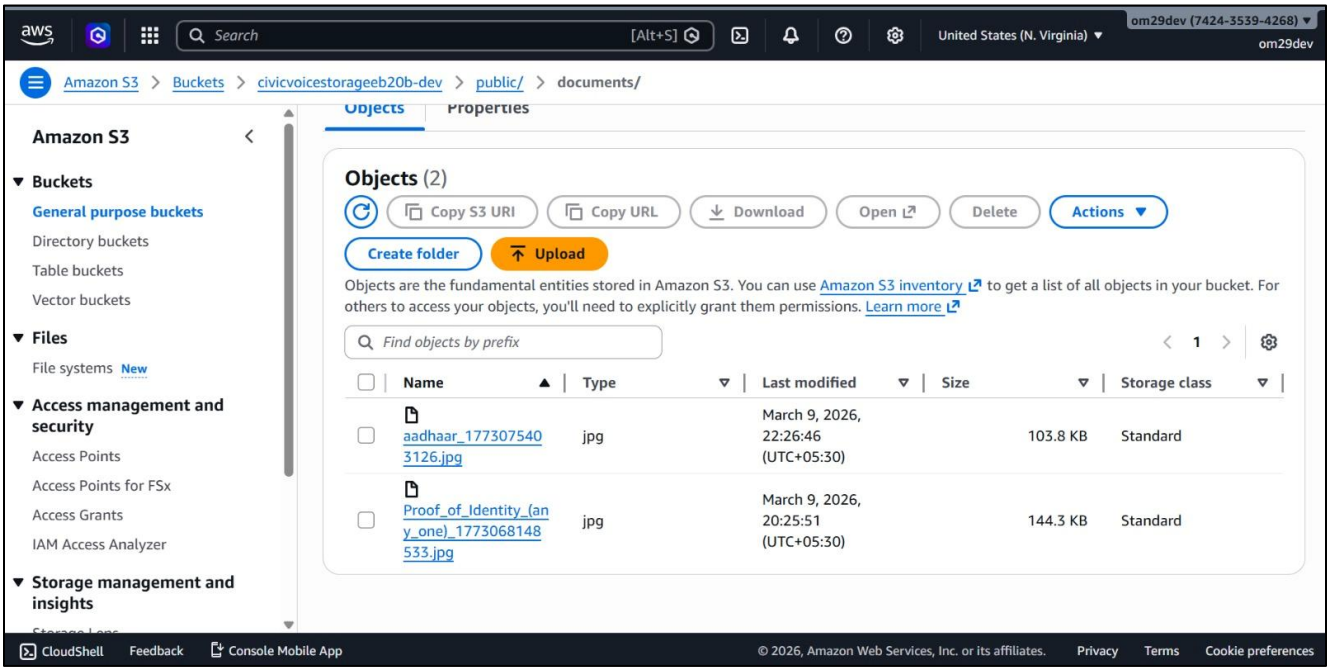
Results and Output

The following four screenshots demonstrate the output of project made using AWS

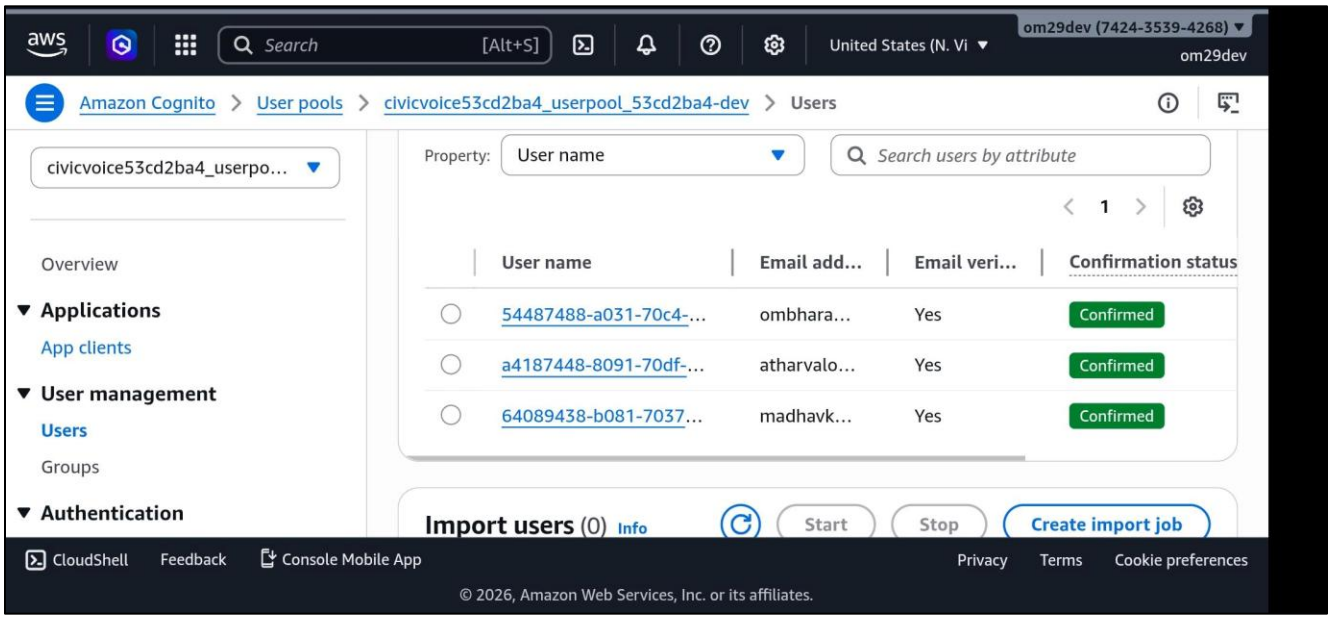
❖ These pictures shows the interface and features & Functions of APP



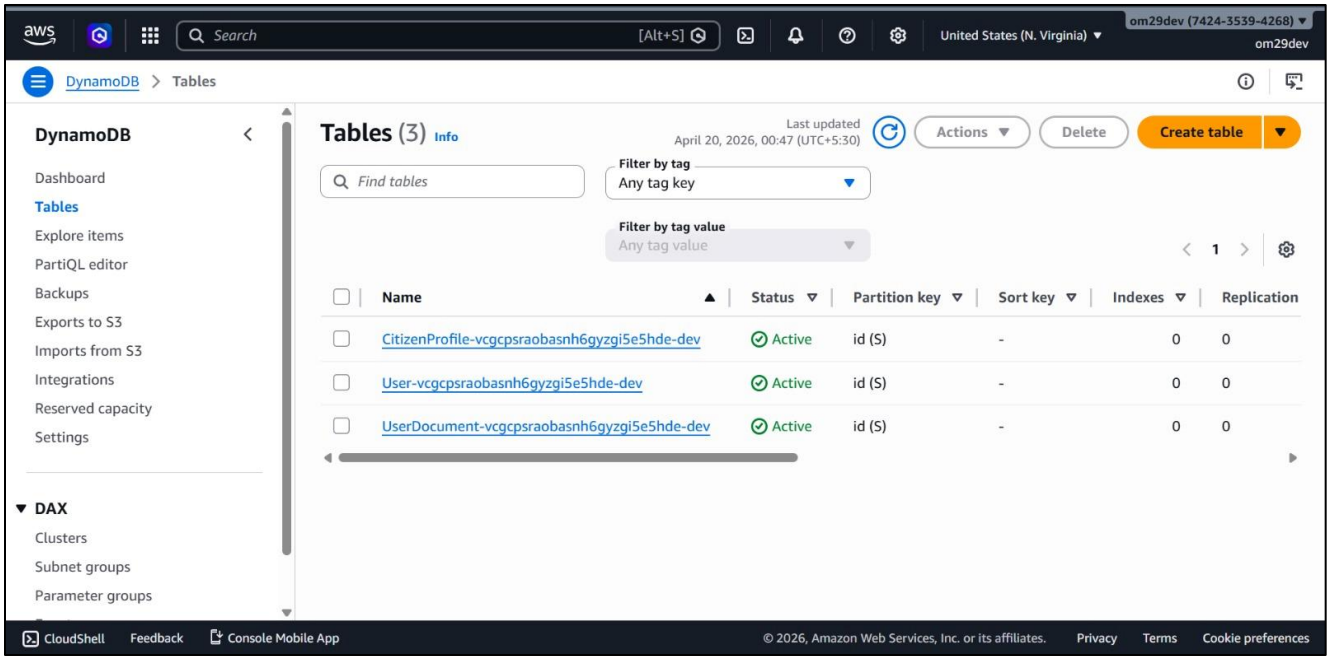
❖ These pictures shows the dashboard of AWS S3 Buckets showing Documents table



❖ These pictures shows the dashboard of AWS Users table



❖ These pictures shows the dashboard of AWS Tables



Conclusion

This mini project successfully demonstrates the design and implementation of a relational database system for the Civic Voice (CVI) platform. The database schema covers five core entities — Citizen, Government Scheme, Service, Document, and Application — modeled using proper ER design and normalized to third normal form (3NF) to eliminate data redundancy.

Through this project, key DBMS concepts were applied in a practical, real-world context: DDL commands (CREATE), DML commands (INSERT, UPDATE, DELETE), complex SELECT queries with multi-table JOINs, aggregate functions (COUNT, AVG, SUM), GROUP BY and HAVING clauses, subqueries, and the creation of VIEWs for reporting.

The learning outcomes include a strong understanding of entity-relationship modeling, SQL query writing, and the practical application of database normalization — skills directly applicable to the development of real-world government technology systems like Civic Voice.

References

- Ramakrishnan, R. & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill.
- Elmasri, R. & Navathe, S. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.
- MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/refman/8.0/en/
- W3Schools SQL Tutorial — https://www.w3schools.com/sql/
- National Informatics Centre (NIC) — https://www.nic.in/
- Digital India Initiative — https://www.digitalindia.gov.in/
- GitHub Repository: Civic Voice (CVI) — https://github.com/om29dev/civic_voice